

Informe Técnico Analítico - “Sistemas de Semáforos Inteligentes”

Integrantes del grupo: Aponte Eduardo Jose

Caceres Lourdes Gabriela

Gómez Susana Nelida

Gonzalez Vanessa Marlene

Los Santos Misael Adonias

Tema investigado: HASKELL

En este documento, dejamos registrado los fundamentos del diseño funcional adoptado por el grupo para la realización del “Sistemas de Semáforos Inteligentes”.

- Nuestros motivos para integrar la librería **cl-json** a nuestro código.
- La bitácora de depuración de cada uno de nosotros
- Las conclusiones que llegamos al reimplementar la funciones **transición** y **timer** al lenguaje Haskell

Introducción al proyecto

El objetivo de este proyecto consiste en el diseño e implementación del núcleo lógico inmutable para un Sistema de Semáforos Inteligentes en intersecciones urbanas críticas. El propósito primordial de este software es proveer una herramienta automatizada de simulación y análisis métrico temporal que optimice el flujo vehicular y la seguridad vial, proporcionando al operador informes estadísticos detallados ante diversos escenarios operacionales.

El sistema está construido bajo el paradigma funcional utilizando el lenguaje ANSI Common Lisp. Para la abstracción y gestión de las constantes, se integra la librería de serialización **cl-json** provista por el ecosistema Quicklisp.

Especificaciones técnicas

El usuario puede interactuar con el sistema a través de una Interfaz de Línea de Comandos (CLI) estructurada mediante un menú jerárquico de opciones iterativo. Las principales herramientas del software se dividen en los siguientes requerimientos específicos:

1. **Modelado y Validación de Transiciones:** Simulación del cambio de luces del semáforo, discriminando secuencias válidas de aquellas inválidas.

2. **Sincronización Temporal (Timer):** Determinación exacta del color activo en un instante de tiempo Unix específico (`timestamp`).
3. **Auditoría Histórica de Eventos:** Registro secuencial de las transiciones operadas en el sistema para la generación de informes cronológicos de ejecución.
4. **Análisis de Optimización de Ciclos:** Cálculo de la duración total (en segundos) de un ciclo completo de luces y su evaluación en relación con los rangos óptimos de la psicología del conductor.
5. **Planificación Temporal de Frecuencias:** Computación del número entero de ciclos íntegros que se ejecutarán dentro de un intervalo parametrizado en minutos.
6. **Análisis Estadístico de Distribución Lumínica:** Generación de un informe de asignación porcentual de tiempo por cada color en un bloque estático de una hora de simulación.

El semáforo inteligente opera bajo una estructura tricolor estándar (Rojo, Amarillo y Verde) cuyos tiempos de permanencia base están asignados en el archivo de configuración `config.json` con los siguientes valores:

- Estado en Rojo (rojo): 90 segundos.
- Estado en Amarillo (amarillo): 6 segundos.
- Estado en Verde (verde): 120 segundos.
- Estado Intermitente (intermitente): 3 segundos de transición de seguridad (incorporado en la Iteración 2).

Se define formalmente como Ciclo Semafórico Completo a la secuencia de transiciones Rojo -Verde - Amarillo - Intermitente. La sincronización cronológica global del sistema se rige mediante el estándar de tiempo Unix (Epoch Time), procesando la entrada como un tipo de dato entero plano para calcular de forma no destructiva el estado activo..

Entorno de Ejecución y Dependencias

Para la correcta ejecución y despliegue del sistema de semáforos, el entorno debe satisfacer los siguientes requisitos:

1. Consola del Sistema (CLI): La interacción se realiza mediante una terminal estándar de comandos (ej. `cmd` en Windows, `bash/zsh` en entornos Unix/Linux).
2. Entorno de Ejecución Common Lisp: Es mandatorio contar con un compilador/intérprete que provea un bucle de lectura, evaluación (REPL). Se recomienda explícitamente el uso de SBCL o CLIPS.
3. Gestión de Dependencias Externas: El sistema requiere la presencia previa del gestor de paquetes Quicklisp configurado en el directorio del usuario. El código fuente de la aplicación automatiza de forma transparente la inicialización del entorno (`setup.lisp`) y la descarga adaptativa de la librería de serialización `:cl-json`.

Fase 1: Diseño e implementación del Sistema de Semáforos Inteligentes (en common LISP)

REQUERIMIENTO 1: Estados de Transición

Objetivo: Modelar y validar de forma determinista el cambio de un estado luminoso a otro, impidiendo secuencias de transición que violen las reglas de seguridad vial.

Funciones que lo integran:

- `transicion (color-actual cambiar-a)`

Conexión entre funciones: Es una función atómica e independiente dentro del módulo core. No invoca funciones auxiliares ni de otros requerimientos.

Naturaleza: Pura. A idénticos argumentos de entrada produce exactamente el mismo valor de retorno, careciendo por completo de efectos secundarios (*side-effects*).

Parámetros de Entrada:

- `color-actual`: Tipo **Símbolo** (SYMBOL). Representa el estado actual del semáforo (Valores válidos: 'en-rojo, 'en-amarillo, 'en-verde).
- `cambiar-a`: Tipo **Símbolo** (SYMBOL). Representa el color destino hacia el cual se intenta transicionar (Valores válidos: 'rojo, 'amarillo, 'verde).

Retorno: Tipo **Lista** (LIST).

- Caso Exitoso: Retorna una lista con dos elementos: el símbolo del estado origen y una cadena descriptiva de la acción, por ejemplo: ('en-rojo "cambiar-a-verde").
- Caso Fallido / Control de Errores: Retorna una lista de control de fallas mediante cláusulas del operador `cond`, por ejemplo: ('error "color-actual-invalido").

REQUERIMIENTO 2: Temporizador Automático (Timer)

Objetivo: Sincronizar el estado activo del semáforo con el tiempo actual (en *Epoch time*) mediante cálculo aritmético modular, permitiendo deducir la luz encendida en cualquier instante temporal sin necesidad de mutar variables de estado.

Funciones que lo integran:

- `calcular-timer (tiempo)`
- `calcular-rem (tiempo rojo verde amarillo intermitente)`
- `compararRem (resto rojo verde amarillo intermitente)`

Conexión entre funciones:

1. `calcular-timer`: Funciona como la interfaz pública del módulo. Valida el tipo de dato de entrada, ejecuta cuatro llamadas a la función externa `obtener-color` para recuperar los tiempos del archivo `.json` e invoca a `calcular-rem`.

2. `calcular-rem`: compara el resto (`rem`) del tiempo Unix con respecto a la duración total del ciclo acumulado (incluyendo los tres periodos de intermitencia de la Iteración 2) y transfiere este valor a `compararRem`.
3. `compararRem`: Evalúa mediante estructuras condicionales el rango temporal al que pertenece el residuo calculado.

Naturaleza: Pura, ya que no altera estados globales ni produce escrituras destructivas.

Parámetros de Entrada:

- `tiempo / resto`: Tipo **Entero** (INTEGER). Representa el timestamp Unix o el residuo `rem` de la división.
- `rojo, verde, amarillo, intermitente`: Tipo **Entero** (INTEGER). Valores en segundos recuperados de la configuración.

Retorno: Tipo **Símbolo** (SYMBOL) o **Cadena** (STRING).

- Retorna un símbolo que representa el estado exacto del semáforo ('en-rojo', 'rojo-intermitente', 'en-verde', 'verde-intermitente', 'en-amarillo', 'amarillo-intermitente) o un string en caso de error estructural.

REQUERIMIENTO 3: Sistema de Auditoría

Objetivo: Capturar, estructurar, formatear y persistir el historial cronológico de cambios de estado del sistema semafórico, proveyendo salidas tanto en la terminal interactiva como en un almacenamiento físico persistente.

Funciones que lo integran:

- `registrar-evento` (timestamp estado-anterior estado-nuevo historial)
- `mostrar-evento` (evento)
- `mostrar-historial` (historial)
- `evento-a-texto` (evento)
- `historial-a-texto` (historial)
- `guardar-informe` (historial)
- `informe` (historial)

Conexión entre funciones: El módulo se divide dos sub-estructuras acopladas por flujos de datos:

- Flujo de Consola: `mostrar-historial` procesa recursivamente la lista del historial extrayendo cada nodo con `car` para enviarlo a `mostrar-evento`.
- Flujo de Persistencia en Disco: `guardar-informe` abre un canal físico de escritura (`with-open-file`), e inyecta el string generado por la función constructora `informe`. Esta última invoca a `historial-a-texto` la cual, mediante recursión constructiva (`concatenate 'string`), une los strings individuales generados por `evento-a-texto` procesando la lista invertida (`reverse`).

Naturaleza: Mixta.

- `registrar-evento`, `evento-a-texto`, `historial-a-texto` e `informe` son **Puras**.

- `mostrar-evento`, `mostrar-historial` y `guardar-informe` son **Impuras**, dado que producen efectos secundarios en el entorno del sistema (I/O por pantalla con `format-t` y mutación de archivos físicos en disco).

Parámetros de Entrada:

- `timestamp`: **Entero** (INTEGER).
- `estado-anterior`, `estado-nuevo`: **Símbolos** (SYMBOL).
- `evento`: **Lista** conteniendo (`timestamp estado-ant estado-nue`).
- `historial`: **Lista de Listas** (LIST de sub-listas) que representa la estructura de datos acumulativa.

Retorno: * `registrar-evento` devuelve una nueva **Lista de Listas** con el nodo insertado en el tope mediante `cons`.

- Las funciones de impresión devuelven **NIL** o cadenas vacías al agotar la recursión.
- Las funciones transformadoras devuelven tipos **Cadena** (STRING).

REQUERIMIENTO 4: Análisis de Ciclos Semafóricos

Objetivo: Calcular el tiempo total en segundos de un ciclo completo de tradiciones del semáforo y emitir una “recomendación” de optimización vial basado en la ingeniería de tráfico y la psicología del conductor.

Funciones que lo integran:

- `duracion` ()
- `recomendacion-ciclo` (`duracion`)

Conexión entre funciones: La salida de la función `duracion` está diseñada para alimentar directamente la entrada de `recomendacion-ciclo`. Ambas dependen internamente de la consulta de constantes mediante `obtener-color`.

Naturaleza: Pura. No mutan variables y operan de forma matemática determinista.

Parámetros de Entrada:

- `duracion`: no recibe datos de entrada como parámetros. Para `recomendación-ciclo`, recibe un tipo **N Numérico / Entero** (INTEGER) que representa el total de segundos de la revolución.

Retorno:

- **Duración:** Un **número entero** (INTEGER) resultado de la suma de todas las luces e intermitencias registradas en el JSON.
- **recomendación-ciclo:** Un mensaje en pantalla (STRING) de la evaluación de la condiciones del rango establecido ("Ciclo demasiado largo", "Ciclo demasiado corto", "Ciclo óptimo"), o interrumpe la ejecución mediante señales de desborde con **error** si las precondiciones tipadas fallan.
-

REQUERIMIENTO 5: Planificación Temporal

Objetivo: Determinar cuantitativamente cuántos ciclos completos tendrán lugar durante un bloque temporal parametrizado por el usuario en minutos.

Funciones que lo integran:

- cantidad-ciclos (minutos)

Conexión entre funciones: Es una función independiente, pero acoplada con el archivo de configuración externo; invoca repetidamente a obtener-color para obtener la duración.

Naturaleza: Pura.

Parámetros de Entrada:

- minutos: Tipo **Entero** (INTEGER) representa el bloque temporal (en minutos).

Retorno: Un **Elemento Único Entero** (INTEGER). Esto se garantiza mediante la aplicación de la función primitiva de truncamiento hacia abajo **floor**, la cual descarta las fracciones de ciclo.

REQUERIMIENTO 6: Informe de Distribución Temporal

Objetivo: Determinar el porcentaje de tiempo que cada luz (de forma individual) permanecerá activa durante un intervalo estándar de una hora (3600 segundos), considerando escenarios de ciclos incompletos (residuos).

Funciones que lo integran:

- porcentaje-color ()
- calcular-rest (resto rojo verde amarillo actual)
- calcular (color resto ciclos)

Conexión entre funciones:

1. porcentaje-color: Funciona como el componente central del flujo. Calcula la duración total de un ciclo, determina cuántos ciclos completos caben en una hora (3600 segundos) y obtiene el residuo de tiempo restante.
2. Flujo de resolución: Mediante composición funcional, la función invoca a calcular-rest para asignar los segundos del ciclo incompleto al color correspondiente (evaluado según las prioridades de la estructura condicional cond).
3. Salida: Finalmente, transfiere tanto el residuo asignado como el total de ciclos completos a la función aritmética calcular para el procesamiento final.

Naturaleza: Pura.

Parámetros de Entrada:

- porcentaje-color: Ninguno (asume la constante de 1 hora del dominio del problema).
- resto, rojo, verde, amarillo, color, ciclos: Tipos **Entero** (INTEGER).
- actual: Tipo **Símbolo** (SYMBOL). Clave de mapeo condicional.

Retorno: Tipo **Lista de Celdas Cons** (`LIST` de `CONS` `pairs`). Devuelve una lista que empareja el símbolo del color con su cálculo porcentual exacto en punto flotante.

Fase 2: Integración del gestor de paquetes Quicklisp - Librería cl-json

En el marco de la Fase 2 del proyecto, y en cumplimiento con el requisito de integrar el gestor de paquetes Quicklisp, se seleccionó la librería externa `cl-json`. Esta decisión de diseño responde a la necesidad de desarticular las constantes temporales del semáforo (parámetros de negocio) de la lógica ejecutable del código. Mediante la externalización de estas variables en el archivo `config.json`, el sistema adquiere flexibilidad y dinamismo, reduciendo el riesgo de dependencia de datos estáticos dispersos en múltiples funciones del sistema. Asimismo, debido a la naturaleza modular y puramente funcional del código base desarrollado en la Fase 1, la integración de este componente presentó un bajo impacto de refactorización, preservando la estructura y comportamiento de las funciones ya validadas.

La manipulación y lectura de esta estructura en el entorno de Lisp se resuelve mediante la implementación de una capa de abstracción de datos compuesta por dos funciones auxiliares complementarias:

1. **leer-config**: Se encarga de la apertura del flujo de entrada físico (`config.json`) mediante el macro orientado a recursos `with-open-file`. Delega en la librería `cl-json:decode-json` la deserialización sintáctica del archivo, transformando los pares clave-valor en una estructura nativa de lista de asociación (*A-list*) de Common Lisp.
2. **obtener-color** : Actúa como la interfaz pública de la lista obtenida del archivo `config`. Recibe como parámetro un símbolo que representa la clave del color (`:ROJO`, `:AMARILLO`, `:VERDE`, `:INTERMITENTE`) y efectúa una búsqueda lineal indexada utilizando el predicado estructural `ASSOC` sobre la *A-list* devuelta por `leer-config`. Para evitar el retorno del par asociativo completo (`CLAVE . VALOR`), se aplica la función selectora `cdr`, abstrayendo al resto del sistema y devolviendo únicamente el elemento entero que representa el tiempo en segundos.

La combinación de ambas funciones transforma el acceso a los datos, permitiendo que el resto de los módulos (como `calcular-timer`, `duracion`, e `informe-porcentaje`) utilicen los valores de forma dinámica y centralizada, ocultando los detalles de la implementación física del archivo y garantizando la modularidad del software.

Fase 3: Reimplementación y comparación con Haskell

Haskell es un lenguaje de programación multipropósito y puramente funcional, reconocido por su tipado estático fuerte y su inmutabilidad de datos. Es una herramienta fundamental para entender nuevos paradigmas de desarrollo, evitando estados mutables y enfocándose en expresar la lógica a través de funciones matemáticas.

Características

- **Funcional Puro**: Las funciones matemáticas no tienen efectos secundarios. La misma entrada siempre producirá exactamente la misma salida.

- Tipado Estático con Inferencia: El compilador detecta errores antes de ejecutar el programa. Además, es muy inteligente deduciendo los tipos de datos, por lo que rara vez necesitas declararlos explícitamente
- Evaluación Perezosa (Lazy Evaluation): Haskell no calcula los valores hasta que realmente los necesita, lo que permite crear estructuras de datos infinitas y mejorar el rendimiento.
- Inmutabilidad: Una vez que se asigna un valor a una variable, esta no puede cambiar, lo que hace al código más seguro y fácil de depurar.

Ventajas

- Código muy conciso y fácil de mantener,
- Excelente manejo de la concurrencia y paralelismo.
- Altamente valorado por la seguridad y prevención de errores en tiempo de compilación.
- El código del software de Haskell es breve, claro y fácil de mantener.
- Menos propensas a errores y ofrecen una gran fiabilidad.
- La brecha semántica entre el programador y el lenguaje es mínima.

Desventajas

- Curva de aprendizaje empinada para quienes vienen de lenguajes como Python o C++.
- Ecosistema menor en comparación con lenguajes de propósito general masivo
- Interfaz con hardware o entrada/salida (I/O) más compleja debido a su pureza matemática.
- Es utilizado por gigantes tecnológicos y corporaciones en criptomonedas, inteligencia artificial, finanzas y desarrollo web.

Diferencia de Haskell con otros lenguajes de programación

Es un lenguaje de programación puramente funcional, Haskell es muy distinto de muchos otros lenguajes. Se diferencia en particular de los lenguajes que se orientan al paradigma imperativo. Los programas escritos en lenguaje imperativo ejecutan secuencias de instrucciones. Incluso durante la ejecución, el estado de estas declaraciones puede cambiar, por ejemplo, al modificar las variables. Las estructuras de flujo de control garantizan que las instrucciones se puedan ejecutar repetidamente.

Reimplementación de las funciones **transición** y **timer** al lenguaje Haskell

La experiencia de aprendizaje con Haskell resultó inicialmente compleja debido a sus diferencias sintácticas respecto de la mayoría de los lenguajes de programación modernos y de uso masivo. Su enfoque puramente funcional introduce conceptos y formas de resolución de problemas que requieren un cambio de paradigma para quienes poseen experiencia previa en lenguajes imperativos u orientados a objetos.

No obstante, se observaron similitudes conceptuales con Common Lisp, especialmente en el uso de funciones puras, la ausencia de efectos secundarios y la importancia de la composición funcional como mecanismo principal para la construcción de soluciones. Estos aspectos facilitaron la comprensión de

algunos conceptos fundamentales, ya que ambos lenguajes promueven un enfoque declarativo y matemático para el desarrollo de programas.

A pesar de estas similitudes, la adaptación a la sintaxis de Haskell y a las herramientas necesarias para su ejecución y desarrollo demandó un tiempo considerable. La configuración del entorno, el sistema de tipos y las particularidades de la escritura de funciones representaron una curva de aprendizaje más pronunciada de lo esperado. En consecuencia, el proceso de implementación resultó igual de demandante y desafiante que en Lisp, aunque permitió adquirir una comprensión más profunda de los principios de la programación funcional.

Common Lisp tiene un tipado dinámico. Haskell tiene un tipado estático estricto. ¿Cómo modelaron los colores del semáforo? Expliquen qué es un Algebraic Data Type (ADT) y cómo el compilador de Haskell impide pasar un estado inválido antes de que el programa se ejecute.

¿Qué es un Algebraic Data Type (ADT)?

Un ADT (**Tipo de Datos Algebraico**) es un tipo de dato personalizado que se crea combinando otros tipos. Se llaman "algebraicos" porque se forman mediante operaciones similares a la suma y la multiplicación algebraicas:

1. **Tipos Suma (unión):** El tipo puede ser una cosa **O** otra.
2. **Tipos Producto (conjunción):** El tipo contiene una cosa **Y** otra.

¿Cómo se modelaron los colores del semáforo en este código?

Los colores y estados de transición se modelaron de forma **implícita** como cadenas de caracteres (String, que en Haskell es un sinónimo de lista de caracteres [Char]) mapeados directamente dentro de las expresiones lógicas de la función `compararRem`. Los tiempos de duración (90, 6, 120, 3) se encuentran acoplados de manera fija (*hardcoded*) en la función de nivel superior `calcuTimer`.

¿Cómo impide el compilador (GHC) pasar un estado inválido?

A diferencia de Common Lisp, donde los errores de tipo o símbolos inválidos (como `color-actual-invalido` o `transicion-no-valida`) se detectan **en tiempo de ejecución** (cuando el programa ya está corriendo y falla en los bloques `cond`), **Haskell opera en tiempo de compilación**.

El compilador de Haskell (GHC) utiliza un mecanismo llamado **Verificación Estática de Tipos y Análisis de Exhaustividad** (*Exhaustiveness Checking*):

- **Seguridad en la Entrad:** Gracias al tipado estático estricto, GHC garantiza la total inmunidad del temporizador contra datos de tiempo corruptos en la entrada. Si otra sección del sistema intenta invocar a `calcuTimer` pasándole un argumento inválido (como un string "14:30" o un flotante 178145.2), el compilador detendrá la construcción del binario inmediatamente con un error de tipo (*Static Type Mismatch*). El programa inválido jamás llegará a ejecutarse.
- **Falla de Seguridad en la Salida:** Debido a que el tipo de retorno es un genérico String, **GHC es completamente incapaz de detectar un estado operativo inválido**. Si hay un error de tipeo (por ejemplo, escribir "en-roho" en vez de "en-rojo"), o si una función de control recibe el resultado de `calcuTimer`, para el compilador cualquier cadena de texto es matemáticamente válida. No hay control semántico en tiempo de compilación sobre el dominio del semáforo.

Cómo afecta la Evaluación Perezosa (Lazy Evaluation) de Haskell al procesamiento de los tiempos del temporizador si tuvieran un flujo infinito de eventos de tráfico.

La evaluación perezosa es una técnica de programación en la que una expresión o una función no se evalúa hasta que no sea necesaria. La expresión se evalúa sólo cuando los resultados se necesitan para una operación en particular, en lugar de evaluarla de inmediato. Esto se usa para mejorar el rendimiento y la eficiencia del programa, al evitar evaluar expresiones innecesarias.

En la mayoría de los lenguajes tradicionales (incluyendo Common Lisp), se utiliza la Evaluación Estricta. Esto significa que cuando pasas una expresión como argumento a una función, el compilador o intérprete la calcula por completo antes de ejecutar el cuerpo de la función.

En cambio, Haskell utiliza Evaluación Perezosa por defecto. Haskell no evalúa una expresión hasta que su resultado sea estrictamente necesario para mostrar una salida o tomar una decisión. En lugar de calcular el valor inmediatamente, el compilador crea una estructura interna llamada "Thunk" (un marcador de posición o promesa de evaluación). El Thunk contiene la fórmula matemática y las referencias necesarias para calcular el valor sólo si y cuando otra parte del programa lo demande.

Si haskell tuviera que procesar un flujo infinito de eventos de tráfico, cada elemento del flujo se almacenará en memoria como un thunk (un puntero a la computación que lo genera) y solamente se van a evaluar las expresiones hasta que sea estrictamente necesario para producir un resultado (como mostrarlo en pantalla).

Bibliografía

- [Haskell MOOC](#)
- [Haskell: el lenguaje de programación funcional](#)
- [¿Por qué Haskell?](#)
- [Curso Haskell 2026](#)
- [aprende haskell - youtube](#)